Computational Thinking Using Python

- Aulas 5 e 6 – Semana 3 – Variáveis , Operadores Lógicos, Aritméticos e de Atribuição

Material produzido pelo Professor
Fabio Souza

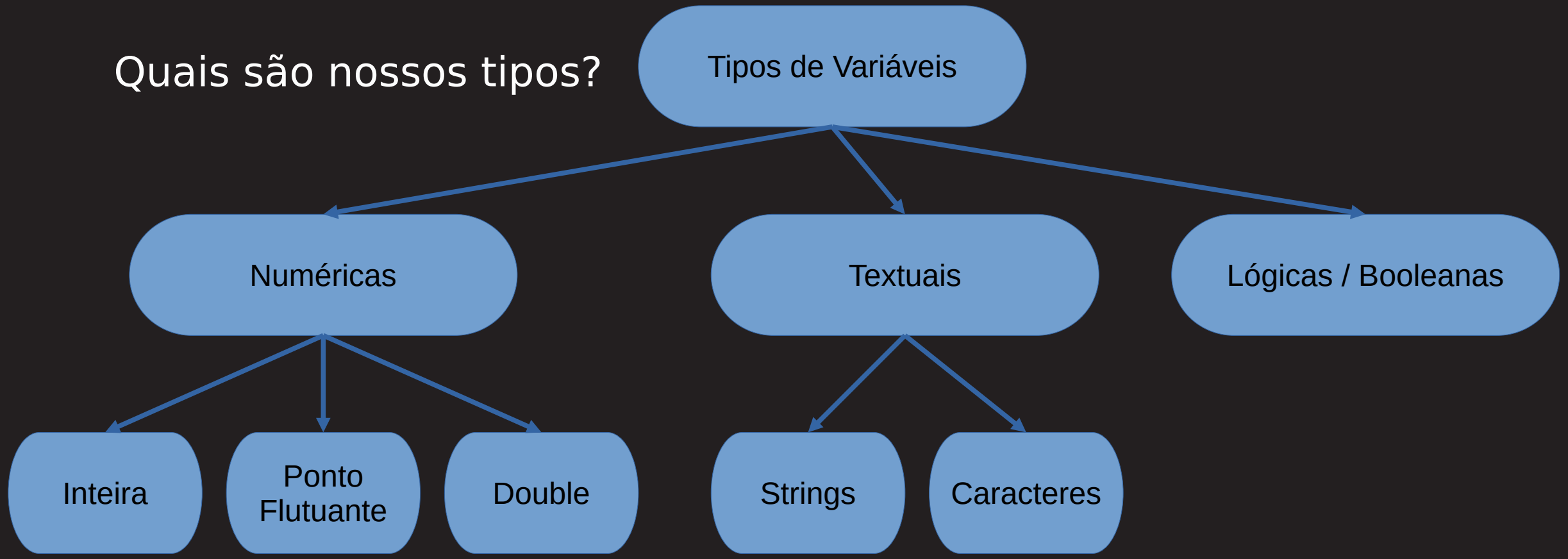
FIAP
GRADUAÇÃO

Variáveis Recordando

Relembrando as variáreis

Tipos de Variáveis Conceito Geral

Quais são nossos tipos?



Tipos de Variáveis – Numéricas

Numéricas

Variáveis do tipo numérica são aquelas que utilizamos para armazenar dados destinados a cálculos e também para representar noções de ordem, intensidade ou conceitos relacionados a essas medidas.

Tipos de Variáveis – Numéricas

Inteiros (int)

Variáveis do tipo numérica inteiras, ou para nós no Python int, são aquelas variáveis dentro da escala dos números inteiros, que são como estes nos exemplos abaixo:

- 4
- 0
- -100

Tipos de Variáveis – Numéricas

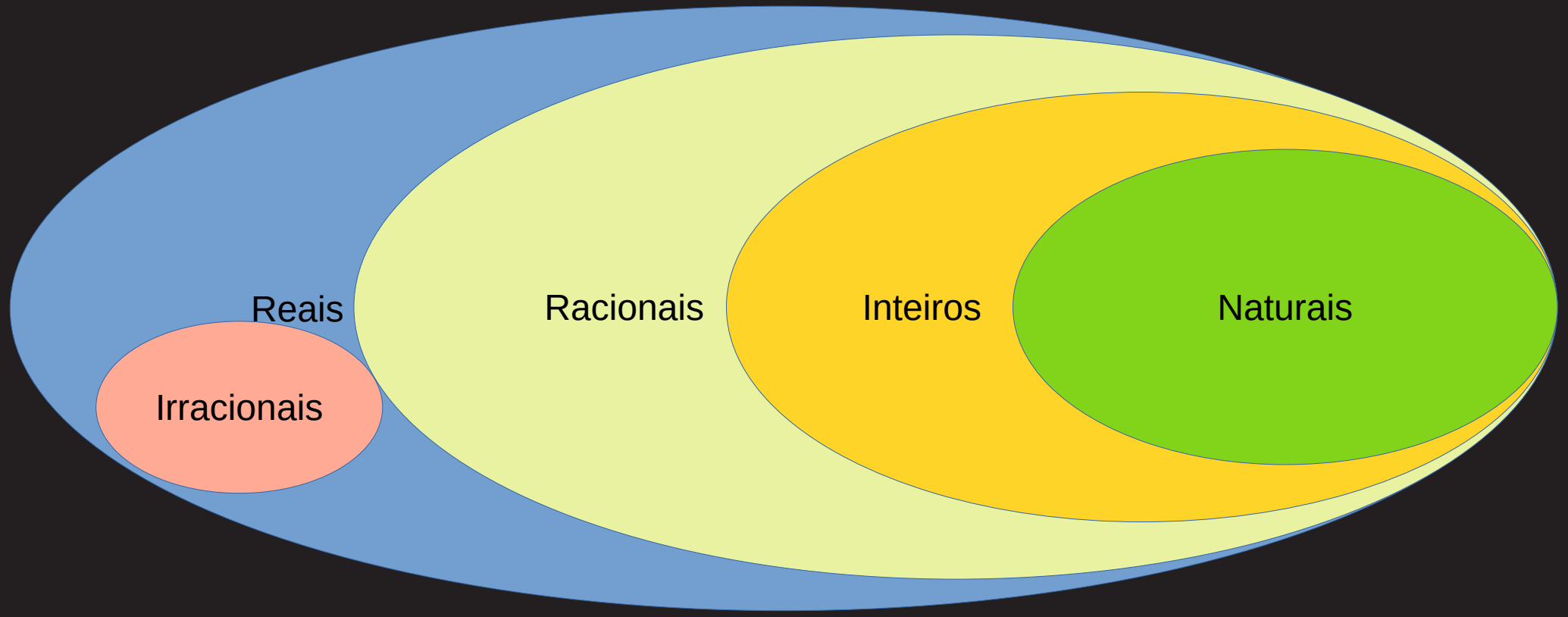
Reais / Ponto Flutuante
(float)

Variáveis do tipo numérica reais, ou para nós no Python float, são aquelas variáveis dentro da escala dos números reais, capazes de armazenar valores decimais (que podem ser infinitos entre dois números inteiros):

- 4,1
- 0,00001
- -100,00000001

Tipos de Variáveis – Numéricas

Para relembrar dos nossos conjuntos de números:



Tipos de Variáveis – Textuais

Textuais / Strings (str)

Variáveis do tipo textuais ou strings, ou para nós no Python str, são aquelas variáveis capazes de armazenar todos os tipos de caracteres alfanuméricos, incluindo números, letras e outros caracteres especiais ($, @ ...) :

- FIAP
- 11 99944-5566
- helpdesk@organizacao.com.br

Tipos de Variáveis – Textuais

Textuais / Strings (str)

Para que o Python, e a vasta maioria das linguagens, identifique que estamos falando de variáveis textuais, precisamos usar as “ ” ou ‘ ‘, desta forma portanto:

- “FIAP”
- ‘11 99944-5566’
- “helpdesk@organizacao.com.br”

Tipos de Variáveis – Textuais

Textuais / Strings (str)

Vamos notar que as strings são sequências de caracteres, e por isso, podemos acessar e manipular cada elemento contigo em uma str! Temos muitos métodos a serem explorados aqui!

Tipos de Variáveis – Lógicas

Lógicas / Booleanas
(bool)

Variáveis do tipo lógicas ou booleanas, ou para nós no Python bool, são aquelas variáveis capazes de armazenar apenas informações lógicas:

- True (Verdadeiro)
- False (Falso)
- 0 e 1 também funcionam como booleanas, afinal, é assim que o computador as enxerga!

Tipos de Variáveis – Lógicas

Lógicas / Booleanas
(bool)

O que devemos lembrar sobre estas variáveis:

- A escrita em Python é *case-sensitive* (diferencia maiúsculas de minúsculas). Para valores lógicos, usamos sempre a primeira letra maiúscula: True e False. Nunca use false ou TRUE, pois o Python não os reconhecerá como booleanos!!

- Zeros, listas vazias e Nones (nulo em Python) também são False, quase todo o resto na linguagem é tratado como True, mas sobre isso discutiremos mais a diante quando falarmos de estruturas de dados e testes condicionais (Truthiness e Falsiness na linguagem)!

Regras na Criação de Variáveis

Regras de Ouro

Regras Gerais para a Criação de Variáveis

Para atribuirmos nomes às variáveis importante (na verdade MUITO importante) que sigamos algumas diretrizes:

- Começar sempre com uma LETRA, como nos exemplos a seguir;
  - rm573312;
  - L1;
  - A_1

Regras Gerais para a Criação de Variáveis

- Nunca com números ou outros caracteres:
  - 573312rm;
  - 1l, 2l, xl ...;
  - _A1;
  - 1_A;

Regras Gerais para a Criação de Variáveis

- Em, Python:
  - O caractere “_” servem para uma atribuição vazia, que o sistema vai ignorar;
  - Os números impedem o interpretador de entender que aquilo é uma variável;

Regras Gerais para a Criação de Variáveis

Mais uma diretriz:

- Nunca colocar caracteres especiais, exceto “_”, nos nomes das variáveis:
  - O certo:
    - aula_1;
    - rm_573312;

Regras Gerais para a Criação de Variáveis

- O errado:
  - aula-1;
  - rm$573312;
  - n@tural;

- Isso, novamente, impede o interpretador de definir os nomes das variáveis! Imagine você tentando colocar uma variável eu+ela/ele e o interpretador conseguir entender que isso é uma variável!!!

Regras Gerais para a Criação de Variáveis

Notas:

- Antigamente, as linguagens eram baseadas no padrão ASCII, que usava apenas 7 bits (128 caracteres). Isso limitava os nomes ao alfabeto inglês, números e o sublinhado (_)!!! Legados do passado...

Regras Gerais para a Criação de Variáveis

Notas:

- Hoje, linguagens modernas como o Python 3 já aceitam Unicode. Isso significa que, tecnicamente, você pode usar caracteres especiais como acentos ou até emojis (Mas.... é recomendado):
  - preço = 10 (Funciona perfeitamente no Python 3)
  - π = 3.14 (Também funciona!)

Regras Gerais para a Criação de Variáveis

Notas:

- Bom porquê de não utilizar caracteres especiais:
  - Portabilidade: Um colega em outro país com um teclado diferente terá dificuldade em editar seu código.
  - Ferramentas: Nem todos os editores de texto lidam bem com codificações complexas, o que pode corromper o arquivo.

Regras Gerais para a Criação de Variáveis

Mais uma diretriz (última da qual vamos falar):

- Podemos utilizar o *camelCase*:
  - valorFinal;
  - MediaSemestral;
- Ou podemos utilizar o *snake_case* (nosso *underscore* entre as palavras):
  - valor_final;
  - media_semestral;

Regras Gerais para a Criação de Variáveis

Assim podemos afirmar que, apesar da comunidade Python, utilizar mais o *snake_case*, aceitaremos ambos como válidos em nossas aulas, uma vez que outras comunidades como Java, JavaScript prefiram o *camelCase*.

- Fora deste padrão, saímos das boas práticas de programação, o que não vamos querer fazer!

Regras Gerais para a Criação de Variáveis

- Legibilidade é tudo! Escolha um se siga com ele!!! Não fiquem variando!

- *PascalCase* (ValorFinal) é para nomes de classes, e vamos falar sobre isso mais adiante!! Portanto, se usarem o *camelCase*, comecem com letra minúscula!!

Relembrando – Operador de Atribuição

Seria a atribuição uma igualdade?

Operador de Atribuição (igualdade?)

Temos

- nomeDaVariavel ← dado;
  - nome ← “João”;
  - valorDoPastel ← 13.0;
  - registroDeMatricula ← 573312

Variáveis - Atribuições

Em Python:

- nomeDaVariavel = dado;
  - nome = “João”;
  - valorDoPastel = 13.0;
  - registroDeMatricula = 573312

Desafio Rápido sobre Variáveis

Teste seus conhecimentos, rapidinho...

Variáveis - Atribuições

Vamos “declarar” duas variáveis aqui, uma usando *camelCase*, e outra usando *snake_case*:

- Declare uma variável para taxa de conversão do dolar, coloque um valor estimado!
- Declare uma variável que vai receber um texto completo, coloque o texto que quiser nela!

Operadores Aritméticos

Contando, somando...

Operadores Aritméticos

Operadores aritméticos, são aqueles que usamos desde a escola, e são os mais usados em nossos códigos:

- São eles:
  - “+” → indicam inclusão, soma, e como veremos, pode ser utilizado até com variáveis não-numéricas;
    - 1 + 2
    - “Segui” + “ “ + “aprendendo”

Operadores Aritméticos

Operadores aritméticos:

- "-" → indicam exclusão, diminuição, só operam com variáveis numéricas;
  - 2 - 1
  - Com texto não funciona!!

Operadores Aritméticos

Operadores aritméticos:

- “*” → indicam multiplicação, podem indicar copiar ‘n’ vezes, e como veremos, pode ser utilizado até com variáveis não-numéricas;
  - 2 + 2
  - “Bravo ” * 3

Operadores Aritméticos

Operadores aritméticos:

- "/" → indicam divisão, partição de algo, só operam com variáveis numéricas;
  - 2 / 2
  - Com texto não funciona!!

Operadores Aritméticos

Outros operadores aritméticos:

- “%” → indicam o RESTO de uma divisão:
  - 10%2 = 0
  - 10%3 = 1;

Operadores Aritméticos

Outros operadores aritméticos:

- “//” → indicam a DIVISÃO INTEIRA entre dois valores:
  - 10//2 = 5
  - 10//3 = 3;

Operadores Aritméticos

Outros operadores aritméticos:

- “**” → indicam a exponenciação entre dois valores:
  - 2**3 = ...;
  - 9**(1/2) = ...;

Desafio Rápido sobre Oreradores Aritméticos

Teste seus conhecimentos, rapidinho...

Operadores Aritméticos

Vamos no exercitar um pouco!!

- Faça estas operações no Python (pode ser no terminal mesmo):
  - 55 – 11;
  - “Pare “ + “ “ + “agora”;
  - “Não pare siga, siga! ” * 5;
  - 88 / 4;
  - 9**(1/3)

Operadores Aritméticos

Agora atribuam estas operações à variáveis, escolha os melhores nomes e executem!!!

Operadores Compatativos

Comparando, testando!

Operadores Comparativos

Operadores comparativos, respostas ao uso de operadores comparativos são sempre “verdadeiro” ou “falso”, como em respostas binárias de “sim” e “não”:

- São eles:
  - “>” → indicam comparação, “maior que”. Este operador compara uma variável à outra onde a hipótese é que o primeiro é MAIOR que o segundo;
    - 2 > 1;
    - 53 > 97;

Operadores Comparativos

Operadores comparativos:

- “<” → indicam comparação, “menor que”. Este operador compara uma variável à outra onde a hipótese é que o primeiro é MENOR que o segundo;
  - 2 < 1;
  - 53 < 97;

Operadores Comparativos

Operadores comparativos:

- “ == ” → indicam comparação, “igual à”. Este operador compara uma variável à outra onde a hipótese é que o primeiro é IGUAL ao segundo;
  - 2 == 1;
  - 53 == 53;

Operadores Comparativos

Operadores comparativos:

- “ != ” → indicam comparação, “diferente de”. Este operador compara uma variável à outra onde a hipótese é que o primeiro é DIFERENTE do segundo;
  - 2 != 1;
  - 53 != 53;

Operadores Comparativos

Operadores lógicos:

- “ >= ” → indicam comparação, “maior OU igual à”. Este operador compara uma variável à outra onde a hipótese é que o primeiro é MAIOR ou IGUAL ao segundo;
  - 2 >= 1;
  - 53 >= 53;

Operadores Comparativos

Operadores comparativos:

- “ <= ” → indicam comparação, “menor OU igual à”. Este operador compara uma variável à outra onde a hipótese é que o primeiro é MENOR ou IGUAL ao segundo;
  - 2 <= 1;
  - 53 <= 53;

Operadores Comparativos

Notas:

- Em algumas linguagens
  - Diferença: Antes do Python 3, o sinal para diferença era <>, e em algumas linguagens (principalmente aquelas mais antigas como Pascal e Delphi) o sinal ainda permanece como o único operador de diferença;

Operadores Comparativos

Notas:

- Em algumas linguagens
  - Atribuição: Em linguagens como Pascal e Delphi o operador de atribuição é o “:=” e não o “=”! Então cuidado!

Desafio Rápido sobre Oreradores Comparativos

Teste seus conhecimentos, vapt-vupt...

Operadores Comparativos

Vamos no exercitar um pouco!!

- Faça estas operações no Python (pode ser no terminal mesmo), obtendo as respostas para as seguintes equações e inequações:
  - 55 < 11;
  - “Pare “ == “Pare “;
  - “15 >= 10;
  - True != False;

Operadores Lógicos

Agora atribuam estas operações à variáveis, escolha os melhores nomes e executem!!!

Operadores Lógicos – Devolta para a Tabela Verdade!

E, ou e o que mais?

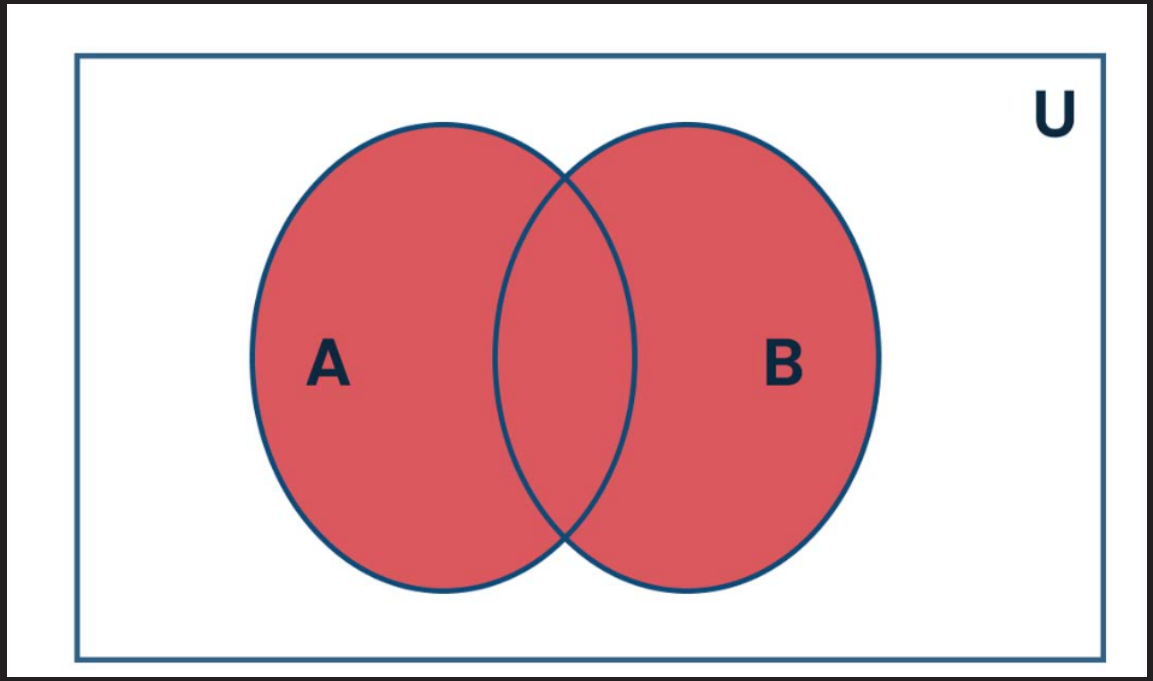
Operadores Lógicos

Operadores lógicos também podem ser “escritos”, e tem um papel significativo em nossos estudos! A tabela verdade é o caminho para resolver este desafio!

- São eles:
  - “ou” ou “or” → indica conjunção, é generoso e pode ser “este ou aquele, podendo serem ambos”. Este operador agrega tudo:
    - True or False;
    - “Python” or “R”

Operadores Lógicos

– “ou” → conjunção pelo diagrama de Venn:



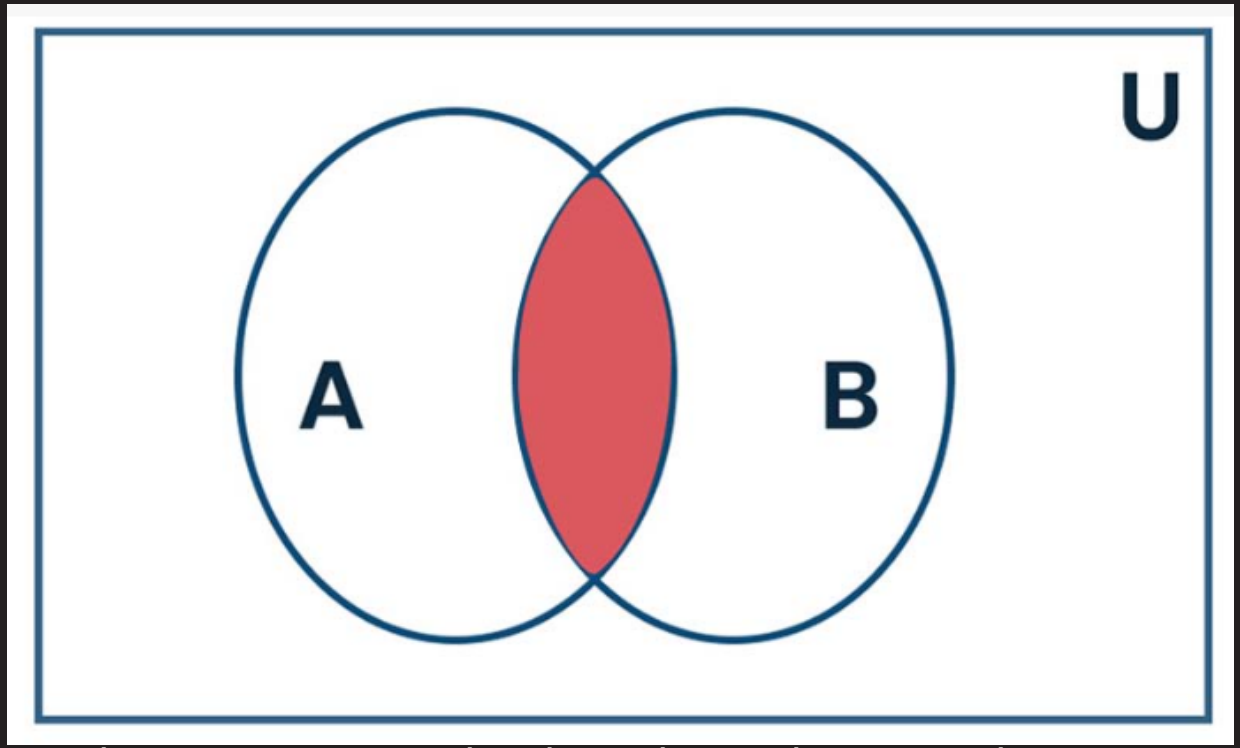
Fonte: https://nebcomputer11.technologychannel.org/number-system/logic-gates/

Operadores Lógicos

- “e”ou “and”→ indica intersecção, é exigente e deve ser “este e aquele, nunca sozinhos, sempre juntos”. Este operador segrega o que não é a intersecção de todos, ou seja, só quanto tudo for verdadeiro o “e” será verdadeiro:
  - True and False;
  - True and True;
  - False and False;

Operadores Lógicos

- “and” → intersecção pelo diagrama de Venn:



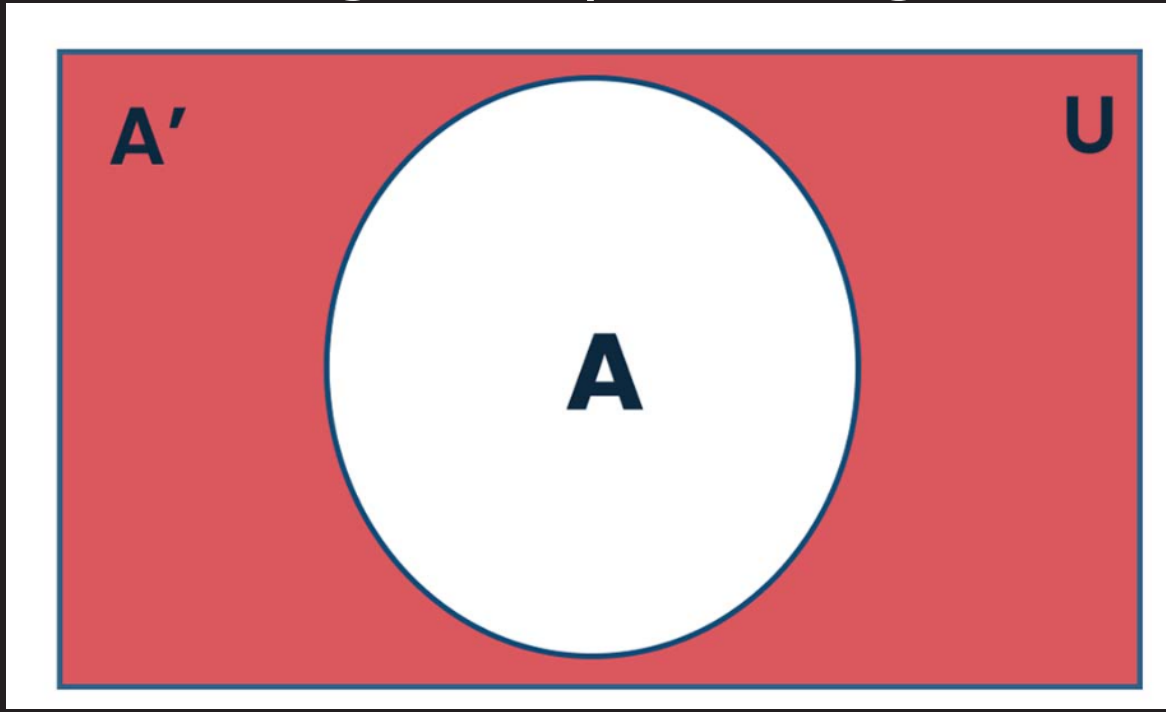
Fonte: https://nebcomputer11.technologychannel.org/number-system/logic-gates/

Operadores Lógicos

- “não”ou “not”→ indica negação, é o “mundo invertido” do nosso estado atual “nada disso, tudo ao contrário”. Este transforma “True” em “False”, vice e versa:
  - not(True);
  - not(False);
  - not(False and False);

Operadores Lógicos

– “not” → negação pelo diagrama de Venn:



Fonte: https://nebcomputer11.technologychannel.org/number-system/logic-gates/

Operadores Lógicos

- Tabela Verdade → A melhor amiga de quem quer entender as respostas de operações lógicas (algebra booleana):

Operadores Lógicos

- Tabela Verdade → “or” (ou):

A	B	A OR B (A ou B)
VERDADEIRO	VERDADEIRO	VERDADEIRO
VERDADEIRO	FALSO	VERDADEIRO
FALSO	VERDADEIRO	VERDADEIRO
FALSO	FALSO	FALSO

Operadores Lógicos

- Tabela Verdade → “and” (e):

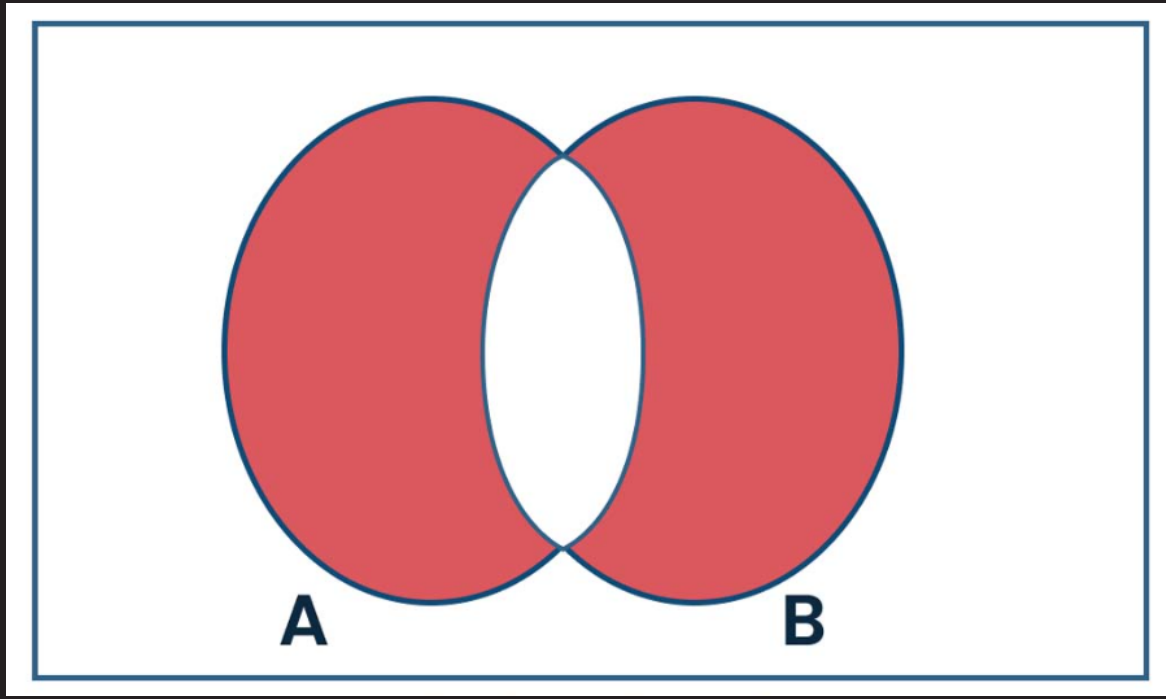
A	B	A and B (A e B)
VERDADEIRO	VERDADEIRO	VERDADEIRO
VERDADEIRO	FALSO	FALSO
FALSO	VERDADEIRO	FALSO
FALSO	FALSO	FALSO

Operadores Lógicos – Menção Honrosa

- “ou exclusivo”ou “xor”→ indica uma condição de conjunção, mas não de ambos ao mesmo tempo, é o “a ou b, nunca a e b”. Ele quebrou o algoritmo do *Perseptron* no final da década de 60 , pois não pode ser resolvido linearmente:
  - A xor B;

Operadores Lógicos

– “xor” → ou exclusivo pelo diagrama de Venn:



Fonte: https://nebcomputer11.technologychannel.org/number-system/logic-gates/

Operadores Lógicos – Menção Honrosa

- Tabela Verdade → “xor” (ou exclusivo):

A	B	A xor B (A ou B, nunca A e B)
VERDADEIRO	VERDADEIRO	FALSO
VERDADEIRO	FALSO	VERDADEIRO
FALSO	VERDADEIRO	VERDADEIRO
FALSO	FALSO	FALSO

Desafio Rápido sobre Oreradores Lógicos

Teste seus conhecimentos, lógica e agilidade!

Operadores Lógicos

Vamos no exercitar um pouco!!

- Faça estas operações no Python (pode ser no terminal mesmo), obtendo as respostas para as seguintes equações e inequações:
  - True or False;
  - 1 and 1;
  - not(True)

Operadores Lógicos

Agora atribuam estas operações à variáveis, escolha os melhores nomes e executem!!!

Resumo

Um resuminho!

Operadores - Resumo

Os operadores aritméticos podem ser:

- Soma (+)
- Subtração (-)
- Multiplicação (*)
- Divisão (/)
- Divisão inteira (//)
- Resto da divisão (módulo) (%)
- Exponenciação (**)

Operadores - Resumo

Os operadores comparativos podem ser:

- Maior que (>)
- Menor que (<)
- Igual à (==)
- Diferente de (!=)
- Maior ou igual à (>=)
- Menor ou igual à (<=)

Operadores - Resumo

Os operadores lógicos podem ser:

- ou (or)
- e (and)
- negação (not())
- E ..... ou exclusivo (xor)

Precedência de Operadores

Quem vem primeiro!

Operadores Aritméticos - Precedência

Os operadores aritméticos tem uma hierarquia entre si, e caso não deixemos isso muito claro (usando os parênteses “()”) estes operadores executam tudo em uma ordem seguindo a seguinte hierarquia:

Operadores Aritméticos - Precedência

1) Parênteses “()”: Tudo o que estiver dentro deles é resolvido primeiro.

2) Exponenciação “**”: Mais forte dentre eles!!

Operadores Aritméticos - Precedência

3) Multiplicação “*”, Divisão “/”, Divisão Inteira “//” e Módulo (resto da divisão) “%”: Todos estes têm a mesma força e vêm antes da soma e subtração!

4) Adição “+” e Subtração “-” : São os últimos da fila!

Operadores Comparativos - Precedência

Os operadores comparativos tem TODOS a MESMA hierarquia, exceto quando estão entre parênteses! Por isso nossa preocupação aqui é == 0!!!!

Operadores Lógicos - Precedência

Os operadores lógicos tem uma precedência bem clara:

1) negação “not()”: Vem antes de tudo!!

2) interseção “and”: O irmão do meio!!

3) conjunção “or”: O mais fraco, sempre age por último!!

Operadores – Ordem de Precedência Geral

Ordem	Categoria	Operadores
1º	Agrupamento	()
2º	Potência	**
3º	Multiplicação / Divisão	*, /, //, %
4º	Soma / Subtração	+, -
5º	Comparação	>, <, >=, <=, ==
6º	Lógico NOT	not
7º	Lógico AND	and
8º	Lógico OR	or

Casting de Variáveis

A revanche!

Variáveis - Casting

Como vimos:

- (int) → (float) → (int)
- (str) → (int) → (str)
- (str) → (float) → (str)
- (str) → (bool) → (str)
- ...

Variáveis - Casting

Como vimos:

- Sobre objetos no Python: A linguagem é fortemente tipada, por isso não conseguimos juntar 1 com “TEXTO” de maneira simples, mas conseguimos forçar que a máquina faça estas operações do jeito correto!

Variáveis - Casting

Como?

- Casting: Como TUDO no Python são objetos as funções que usamos são construtores de classe!! Isso mesmo! Assim criamos novos objetos, com base no anterior!

Variáveis - Casting

Exemplos:

- Casting de string (texto) para integer (inteiro):
  - varInt1 = int(“10”);

Variáveis - Casting

Exemplos:

- Casting de string (texto) para float (real):
  - varFloat = float(“10.5”);

Variáveis - Casting

Exemplos:

- Casting de string (texto) para bool (boleano):
  - varBool = bool(“True”);

Variáveis - Casting

Exemplos:

- Casting de integer (inteiro) para float (real), e o reverso:
  - varFloat2 = float(10);
  - varInt2 = int(10.0);

Desafio Casting de Variáveis

Teste seus conhecimentos nestas transformações!

Variáveis – Desafio

Vamos atribuir algumas variáveis no Python!?!

- Façam o *casting* das variáveis abaixo, testando as formas possíveis de mudança e armazenando em variáveis:
  - varInt1 = 1;
  - VarFloat1 = 1.0;
  - varText = “True”;
  - varBool = False;

Desafio Final

Teste seus conhecimentos contra este divertido desafio!

Variáveis – Desafio

Tente adivinhar o resultado desta expressão:

- False and True or not(False)

Variáveis – Desafio

Como faço isso inverter a resposta com uso de parênteses??

Variáveis – Desafio

E se eu fizer uma divisão das tarefas aqui??

- False and True or not(False)

Como faço isso inverter a resposta com uso de parênteses??